

Cerveau autonome zêta — 23 points

Base de connaissance vivante, montée en compétence, sauvegarde, réplication et reprise catastrophe du projet hprzeta

Auteur : hprzeta — Date : 26 mai 2026

Ce document formalise l'extension logique de Recovery Zeta : transformer le projet en système apprenant par mémoire structurée, Obsidian/Markdown, Git, RAG, backups, synchronisation et reprise automatique sur un système de secours.

Limite réaliste : un système ne peut pas détecter sa propre panne totale depuis la machine morte. Il faut toujours un observateur externe : Raspberry Pi, NAS, mini-PC, VPS ou cloud privé.

1. Créer un agent spécialisé “Knowledge Brain”

Il faut ajouter un agent dédié : hprzeta-knowledge-steward-agent, ou hprzeta-brain-agent. Sa mission est de maintenir la base de connaissances, les formules, les théorèmes, les lemmes, les erreurs, les corrections validées, les dépendances, les limites matérielles/logicielles et les scénarios catastrophe.

Cet agent ne doit pas inventer : il synthétise ce qui est observé dans les commits, rapports, crash reports et résultats validés.

2. Obsidian comme cerveau Markdown

Obsidian est pertinent car il repose sur des fichiers Markdown locaux. Le plugin Obsidian Git permet l'auto commit/sync, l'auto-pull au démarrage, l'historique et les diff dans le vault. Sources : [turn8search155], [turn8search156].

Le vault peut être versionné par Git, indexé par RAG, sauvegardé par Restic et synchronisé par Syncthing. MCPVault peut aussi exposer un vault Obsidian à des assistants IA via MCP, avec recherche, tags, statistiques et intégration Git CLI. Source : [turn8search160].

3. Séparer mémoire, sauvegarde et reprise

Le système doit être découpé en trois couches : mémoire projet, sauvegarde/réplication, et reprise automatique sur une autre machine.

Couche	Briques	Rôle
A — Mémoire projet	Obsidian, Markdown, Git, RAG	Conserver ce que le projet apprend et valide.
B — Sauvegarde/réplication	Git, Restic, Syncthing, dépôt distant, stockage objet	Assurer la survie des données.
C — Reprise catastrophe	Raspberry Pi, VPS, NAS, mini-PC, cloud privé	Détecter la panne et restaurer ailleurs.

4. Architecture recommandée du cerveau hprzeta

Créer un vault de connaissance séparé du dépôt de code, mais synchronisé avec lui.

```
~/hprzeta-lab/12_brain_vault/  
00_dashboard/  
  index.md  
  project_status.md  
  open_questions.md  
  next_actions.md  
01_science/  
  zeta_function.md  
  riemann_siegel.md  
  hardy_z.md  
  turing_method.md  
  li_keiper.md  
  prime_number_theorem.md  
  l_functions.md  
02_formulas/  
  theta_function.md  
  hardy_z_formula.md  
  zero_counting_NT.md  
  explicit_formula.md  
  error_bounds.md  
03_theorems_lemmas/  
  riemann_hypothesis_statement.md  
  argument_principle.md  
  functional_equation.md  
  turing_method_lemmas.md  
  interval_arithmetic_principles.md  
04_numerical_methods/  
  mpmath_method.md  
  pari_gp_method.md  
  flint_arb_method.md  
  sage_method.md  
  comparison_protocol.md  
05_experiments/  
06_errors_lessons/  
07_decisions/  
08_dependencies/  
09_recovery/  
10_training_guide/  
11_daily_logs/
```

5. Missions précises du Knowledge Steward

Après chaque session, l'agent lit nouveaux commits, nouveaux rapports et crash reports ; extrait erreurs, solutions, limites et découvertes ; met à jour les fiches Obsidian ; met à jour formules, théorèmes, dépendances, scénarios catastrophe, dashboard et guide de formation.

Il doit marquer chaque connaissance avec un statut : validé, expérimental, à vérifier, obsolète.

6. Templates Obsidian utiles

Template erreur

```
---
type: error
code: E003_MEMORY_RAM
date: 2026-05-26
status: solved
agent: hprzeta-recovery-agent
tags: [error, memory, recovery]
---

# E003_MEMORY_RAM — Saturation RAM

## Symptôme
Le système ralentit, swap élevé, Ollama + navigateur + VS Code ouverts.

## Cause
Trop de processus lourds pour 8 Go RAM.

## Correction validée
- arrêter modèle lourd
- passer à qwen2.5-coder:1.5b
- relancer test ciblé uniquement

## À ne pas refaire
Ne pas lancer OpenHands + Ollama 7B + navigateur simultanément.

## Liens
- [[token_budget_policy]]
- [[local_model_limits]]
- [[recovery_policy]]
```

Template formule

```
---
type: formula
domain: zeta
status: active
tags: [zeta, formula, riemann-siegel]
---

# Hardy Z(t)

## Formule

$$Z(t) = \exp(i \theta(t)) \zeta(1/2 + it)$$


## Utilisation dans le projet
Utilisée pour détecter les changements de signe sur la ligne critique.

## Implémentations
- Python/mpmath : src/hprzeta/zeta/hardy.py
- Tests : tests/reference/test_hardy.py

## Points de vigilance
- précision de  $\theta(t)$ 
- stabilité numérique
- comparaison aux zéros de référence
```

Template décision technique

```
---
type: ADR
status: accepted
date: 2026-05-26
tags: [architecture, model, local]
---

# ADR — Choix des modèles locaux

## Contexte
PC ASUS i7, 8 Go RAM, GTX 960 4 Go VRAM.

## Décision
Utiliser qwen2.5-coder:1.5b et qwen2.5-coder:3b comme modèles principaux.

## Alternatives refusées
- Qwen 30B/35B localement : trop lourd
- OpenHands permanent : trop gourmand
```

7. Apprentissage réaliste du système

Le système “apprend” sans fine-tuning : erreur rencontrée → diagnostic → solution validée → fiche Obsidian → mise à jour RAG → mise à jour prompt/règle → future session évite l’erreur.

Recovery Zeta fournit déjà la base : error_memory.jsonl, taxonomie d’erreurs, prompts de reprise, anti-répétition, checkpoints et crash reports. Source interne : recovery_zeta.pdf [turn8search1].

8. Architecture de réplication catastrophe

Appliquer une stratégie 3-2-1 : trois copies, deux supports différents, une copie hors site.

Copie	Emplacement	Contenu
1	PC ASUS	Dépôt de travail, vault, RAG, résultats locaux.
2	Raspberry Pi / NAS local	Miroir Syncthing + backups Restic.
3	Cloud privé / dépôt distant / stockage objet	GitHub/GitLab privé + Restic S3/MinIO/Backblaze/VPs.

Restic fournit des sauvegardes chiffrées, dédupliquées et incrémentales vers plusieurs backends dont local, SFTP, S3/MinIO, Backblaze B2, Azure et Google Cloud Storage. Sources : [turn8search150], [turn8search151].

9. Rôle possible du Raspberry Pi

Le Raspberry Pi peut servir d'observateur externe, serveur Wake-on-LAN, collecteur de sauvegardes, miroir Syncthing, déclencheur d'alerte et petit dashboard.

Un Raspberry Pi peut envoyer des paquets Wake-on-LAN à un PC compatible via des outils comme etherwake ou wakeonlan. Sources : [turn8search167], [turn8search170].

10. Détection par Raspberry Pi

Le Pi peut interroger régulièrement l'ASUS : ping, SSH, état de service, mémoire, disque, Git, endpoint santé local. Après plusieurs échecs, il envoie Wake-on-LAN, classe la panne, déclenche backup/restore ou alerte.

```
Toutes les 1 à 5 minutes :
- ping ASUS
- ssh ASUS "systemctl is-active hprzeta-agent"
- ssh ASUS "free -h"
- ssh ASUS "df -h"
- ssh ASUS "git status"
- curl http://asus:8765/health
```

```
Si 3 échecs :
- envoyer Wake-on-LAN
- si pas de retour : scénario ASUS_DOWN
- préparer restauration sur secours
```

Healthchecks permet de surveiller cron jobs et tâches planifiées via pings HTTP ; si le ping n'arrive pas à temps, une alerte est envoyée. Sources : [turn8search173], [turn8search174].

11. Scénarios catastrophe réalistes

Scénario ASUS allumé mais agent bloqué : le Pi détecte absence de heartbeat, tente SSH, lance diagnostic distant, récupère logs et déclenche recovery_agent.

Scénario ASUS éteint mais matériel OK : le Pi détecte absence réseau, envoie Wake-on-LAN, le service systemd hprzeta-resume démarre, lit latest_checkpoint et reprend contrôle.

Scénario ASUS ne redémarre pas : le Pi classe ASUS_DOWN, récupère dernière copie Syncthing/Restic, restaure sur machine de secours ou VPS, clone GitHub, restaure vault et recovery.

Scénario disque mort : nouvelle machine, git clone, restic restore latest, restauration brain_vault/recovery/data_reference, bootstrap script et tests de validation.

12. Nécessité d'une machine de secours

Le Raspberry Pi seul ne remplacera pas le PC ASUS pour l'IA locale ou les calculs zêta lourds. Il orchestre, surveille, réveille, alerte et restaure, mais la reprise de calcul sérieuse demande un mini-PC, un VPS ou un cloud GPU.

Option	Réaliste pour	Limites
Raspberry Pi	Monitoring, Git, backup, scripts légers	Pas de LLM local correct ni calcul lourd.
Mini-PC	Secours local autonome	Coût matériel, maintenance.
VPS/cloud privé	Restauration, CI, RAG léger, agents CPU	Coût mensuel, confidentialité selon hébergeur.
Cloud GPU ponctuel	LLM plus gros, calcul parallèle	Coût et dépendance externe.

13. Cloud privé : solution souvent plus réaliste

La meilleure architecture hybride : PC ASUS pour développement principal, Raspberry Pi pour watchdog local, VPS/cloud privé pour secours, GitHub/GitLab privé pour code, Restic vers S3/MinIO pour backups, Obsidian Git pour cerveau et Syncthing pour réplication locale.

MinIO fournit un stockage objet S3-compatible auto-hébergé, utilisable avec des outils compatibles S3 et des solutions de backup. Sources : [turn8search161], [turn8search162].

14. Service heartbeat du projet

Chaque machine active doit écrire un fichier status.json surveillé par le Pi ou le VPS.

```
~/hprzeta-lab/13_heartbeat/status.json

{
  "machine": "asus-i7",
```

```

"timestamp": "2026-05-26T15:01:00+02:00",
"project": "hprzeta",
"branch": "feature/first-zeros",
"last_commit": "abc123",
"last_checkpoint": "2026-05-26_14-55-10",
"ram_used": "6.1G/8G",
"swap_used": "2.3G/16G",
"disk_free": "742G",
"agent_status": "idle",
"last_successful_test": "tests/reference/test_first_zeros.py",
"next_action": "continue exp_002_hardy_z"
}

```

15. Services systemd à créer

Sur le PC ASUS : hprzeta-heartbeat.service, hprzeta-backup.timer, hprzeta-brain-sync.timer, hprzeta-recovery-watch.service.

Sur Raspberry Pi : hprzeta-watchdog.service, hprzeta-wol.service, hprzeta-mirror-sync.service, hprzeta-alert.service.

Sur VPS/cloud privé : hprzeta-standby.service, hprzeta-restore-test.timer, hprzeta-healthchecks.service.

16. Prompt système du Knowledge Steward

Tu es hprzeta-knowledge-steward-agent.

Mission : construire, maintenir et améliorer la base de connaissances vivante du projet hprzeta.

Tu dois :

1. Lire les nouveaux rapports.
2. Lire les nouveaux crash reports.
3. Lire les nouveaux commits.
4. Extraire les erreurs, solutions, limites et découvertes.
5. Mettre à jour le vault Obsidian Markdown.
6. Mettre à jour les fiches formules, théorèmes, lemmes et méthodes.
7. Mettre à jour les dépendances logicielles.
8. Mettre à jour les scénarios catastrophe.
9. Générer un guide de formation progressif.
10. Ne jamais inventer une connaissance non vérifiée.
11. Marquer : validé, expérimental, à vérifier, obsolète.
12. Préparer la reprise sur une machine vierge.

Livrables : résumé journalier, fiche apprentissage, fiche erreur, dashboard, restore_runbook.

17. Prompt système Disaster Recovery

Tu es hprzeta-disaster-recovery-agent.

Mission : permettre de redémarrer le projet hprzeta sur une autre machine après panne grave.

Tu dois vérifier :

1. Le dépôt Git est récupérable.
2. Les sauvegardes Restic sont valides.
3. Le vault Obsidian est restaurable.
4. Les index RAG peuvent être reconstruits.
5. Les dépendances sont listées.
6. Les modèles Ollama nécessaires sont listés.
7. Les scripts bootstrap fonctionnent.
8. Les tests minimaux passent après restauration.

Tu ne dois jamais supposer qu'une sauvegarde est bonne.

Tu dois périodiquement effectuer un test de restauration.

18. Fichiers essentiels pour redémarrer ailleurs

Le dépôt doit contenir les fichiers nécessaires pour reconstruire l'environnement depuis zéro.

```

README_RESTORE.md
bootstrap_ubuntu24.sh
requirements-minimal.txt
apt-packages.txt
ollama-models.txt
.env.example
docker-compose.recovery.yml
RESTORE_CHECKLIST.md
DISASTER_SCENARIOS.md

```

```

# ollama-models.txt
qwen2.5-coder:1.5b
qwen2.5-coder:3b
llama3.2:3b

```

Exemple RESTORE_CHECKLIST.md

Restore Checklist hprzeta

1. Installer Ubuntu 24.04.
2. Installer Git.
3. Cloner le dépôt.
4. Lancer bootstrap_ubuntu24.sh.
5. Restaurer Restic.
6. Vérifier vault Obsidian.
7. Reconstruire RAG.
8. Télécharger modèles Ollama.
9. Lancer pytest minimal.
10. Lire latest_checkpoint.
11. Relancer hprzeta-recovery-agent.

19. Cycle d'apprentissage autonome

Le cycle complet : expérience → logs → rapport → crash/success report → recovery agent → brain agent → fiche Obsidian → RAG refresh → backup Restic → sync Git/Syncthing → test de restauration → système plus compétent.

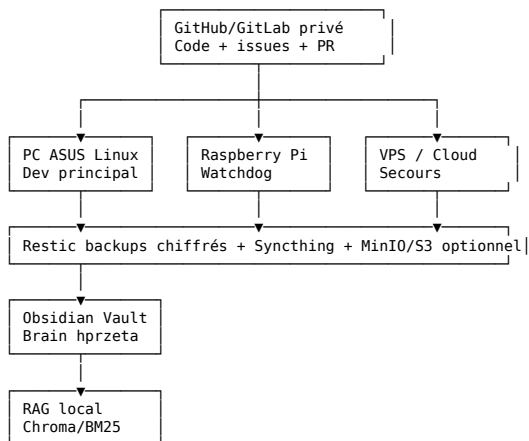
20. Réalisme global

Réaliste à 80-90 % : mémoire projet vivante, Obsidian/Markdown/Git, RAG local, backups Restic, réplication Syncthing, watchdog Raspberry Pi, Wake-on-LAN, alertes, redémarrage logiciel, restauration sur autre machine.

Partiellement réaliste : continuer automatiquement les calculs sans validation humaine, relancer OpenHands sur machine de secours, migrer automatiquement vers cloud GPU, corriger seul des erreurs mathématiques complexes.

Non réaliste sans externe : détecter une panne totale du PC depuis le PC lui-même, survivre à un disque mort sans sauvegarde externe, continuer IA locale lourde sur Raspberry Pi, garantir zéro intervention humaine en panne matérielle grave.

21. Architecture finale recommandée



22. Sécurité électrique et domotique

Pour la domotique, privilégier des solutions sûres : Wake-on-LAN, prise connectée certifiée, UPS, Raspberry Pi, scripts réseau. Ne pas bricoler directement le secteur 230 V avec relais maison sans compétence/habilitation.

Le Raspberry Pi peut être un superviseur efficace sans manipuler dangereusement l'alimentation secteur.

23. Conclusion

L'idée est bonne et constitue l'extension logique de Recovery Zeta. Il faut ajouter deux agents : hprzeta-knowledge-steward-agent et hprzeta-disaster-recovery-agent.

Les trois briques techniques prioritaires : Obsidian vault versionné Git ; Restic + Syncthing + stockage distant ; Raspberry Pi ou VPS watchdog.

Phrase clé : le cerveau du projet ne doit pas seulement contenir du code ; il doit contenir la mémoire de ce que le projet a compris, raté, corrigé, validé, refusé, appris, et ce qu'il faut faire pour renaître ailleurs.

Sources citées

- Recovery Zeta interne : checkpoints, error_memory, recovery agent — [turn8search1]
- Obsidian Git plugin : auto commit/sync, auto-pull, historique, diff — [turn8search155 / turn8search156]
- MCPVault : serveur MCP pour Obsidian vaults — [turn8search160]
- Syncthing : synchronisation continue décentralisée et sécurisée — [turn8search143 / turn8search144 / turn8search145]
- Restic : backups chiffrés, dédupliques, snapshots et restore — [turn8search150 / turn8search151 / turn8search152]
- Raspberry Pi Wake-on-LAN et supervision — [turn8search167 / turn8search170 / turn8search171]
- MinIO : stockage objet S3-compatible auto-hébergé — [turn8search161 / turn8search162]
- Healthchecks : monitoring de jobs cron par pings — [turn8search173 / turn8search174]